

LSTM & GRU: Concepts, Equations, and a Worked Example

Contents

Part 1 — Intuition: LSTM and GRU	1
LSTM	1
GRU	2
Part 2 — One Complete LSTM Time Step	3
Part 3 — Worked Numerical Example	4
1. Starting values	4
2. Step 1 — Forget Gate	4
3. Step 2 — Input Gate	5
4. What happened here?	5
5. Step 3 — Update the Cell State	6
6. Stop here and understand what happened	6
7. Step 4 — Output Gate	7
8. Apply the Output Gate	7
9. Complete calculation in one place	8
10. The entire process, visually	8
11. What each number means	9
12. The key distinction	9
13. One-line memory trick	9

Part 1 — Intuition: LSTM and GRU

LSTM

You have:

Long-term notebook — Cell state: C_t

Short-term working area — Hidden state: h_t

And three assistants:

- **Forget assistant** — “*This old information isn’t useful anymore.*”
- **Input assistant** — “*This new information is important.*”
- **Output assistant** — “*This is the information we should currently show.*”

That’s LSTM.

GRU

Instead of three assistants and two memory systems, you have a simpler mechanism:

- **Reset gate** — “*How much old information should I use?*”
- **Update gate** — “*How much should I keep versus replace?*”

That’s GRU.

Part 2 — One Complete LSTM Time Step

This is the part meant to be understood thoroughly.

At time t , we have as inputs:

$$x_t, h_{t-1}, C_{t-1}$$

Step 1 — Forget

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Then:

$$f_t \odot C_{t-1}$$

means: **decide what old memory survives.**

Step 2 — Input

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

means: **decide how much new information to allow.**

Step 3 — Candidate

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

means: **generate possible new memory.**

Step 4 — Update

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

means: **retained old memory + selected new memory.**

Step 5 — Output

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

means: **decide what memory should be exposed.**

Step 6 — Hidden state

$$h_t = o_t \odot \tanh(C_t)$$

means: **produce the current working representation.**

Part 3 — Worked Numerical Example

Let's work through **one complete LSTM time step numerically**, following the flow: **Forget** → **Input/Candidate** → **Cell State** → **Output** → **Hidden State**.

We'll use very small vectors so every calculation is easy.

1. Starting values

At time t , assume we already have the previous states.

Previous cell state:

$$C_{t-1} = [0.6, 0.8]$$

We won't need the previous hidden state's actual numerical value because we'll **assume the gate calculations have already produced their values**. Suppose the gates/candidate give us:

Forget gate:

$$f_t = [0.5, 0.25]$$

Input gate:

$$i_t = [0.4, 0.5]$$

Candidate memory:

$$\tilde{C}_t = [0.5, -0.2]$$

Output gate:

$$o_t = [0.7, 0.6]$$

These are simple illustrative values.

2. Step 1 — Forget Gate

The forget gate decides:

How much of the old cell state should we keep?

The calculation is:

$$f_t \odot C_{t-1}$$

We have $f_t = [0.5, 0.25]$ and $C_{t-1} = [0.6, 0.8]$.

Element-wise multiplication:

$$[0.5, 0.25] \odot [0.6, 0.8]$$

Calculate each element:

$$0.5 \times 0.6 = 0.3$$

$$0.25 \times 0.8 = 0.2$$

Therefore:

$$f_t \odot C_{t-1} = [0.3, 0.2]$$

Interpretation. Originally:

Old memory = [0.6, 0.8]

The forget gate says:

Keep 50% of first value

Keep 25% of second value

So:

[0.6, 0.8]

↓ forget gate

[0.3, 0.2]

This is the same element-wise forget mechanism illustrated numerically.

3. Step 2 — Input Gate

Now the LSTM asks:

How much new information should we add?

We have $i_t = [0.4, 0.5]$ and candidate memory $\tilde{C}_t = [0.5, -0.2]$.

The calculation is:

$$i_t \odot \tilde{C}_t$$

$$[0.4, 0.5] \odot [0.5, -0.2]$$

Calculate:

$$0.4 \times 0.5 = 0.2$$

$$0.5 \times (-0.2) = -0.1$$

So:

$$i_t \odot \tilde{C}_t = [0.2, -0.1]$$

4. What happened here?

The **candidate memory** says:

Possible new information = [0.5, -0.2]

The **input gate** says:

How much should I allow? = [0.4, 0.5]

Therefore:

$$\begin{array}{ccc} \text{Candidate} & & \text{Input gate} \\ [0.5, -0.2] & \times & [0.4, 0.5] \\ \downarrow & & \\ [0.2, -0.1] & & \end{array}$$

The input gate controls how much of the candidate information needs to be added, while the candidate represents the candidate memory itself.

5. Step 3 — Update the Cell State

Now comes the **most important LSTM equation**:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

We already calculated both pieces.

Retained old memory: $[0.3, 0.2]$

New information: $[0.2, -0.1]$

Therefore:

$$C_t = [0.3, 0.2] + [0.2, -0.1]$$

Element-wise addition:

$$0.3 + 0.2 = 0.5$$

$$0.2 + (-0.1) = 0.1$$

Therefore:

$$C_t = [0.5, 0.1]$$

6. Stop here and understand what happened

This is the entire **memory update**:

Previous memory

$$C_{t-1} = [0.6, 0.8]$$

|

| × Forget Gate

↓

$$[0.3, 0.2] \leftarrow \text{old information retained}$$

|

|

$$+ \leftarrow [0.2, -0.1]$$

↑

Input × Candidate

↓

New memory

$$C_t = [0.5, 0.1]$$

So the LSTM has performed:

$$\text{New Memory} = \text{Retained Old Memory} + \text{Selected New Information}$$

This is the central idea behind the cell-state update.

7. Step 4 — Output Gate

Now we have:

$$C_t = [0.5, 0.1]$$

But the LSTM doesn't simply output C_t . First, apply $\tanh$:

$$\tanh(C_t)$$

Therefore:

$$\tanh([0.5, 0.1])$$

Approximately:

$$[0.4621, 0.0997]$$

So:

$$\boxed{\tanh(C_t) = [0.4621, 0.0997]}$$

8. Apply the Output Gate

We assumed:

$$o_t = [0.7, 0.6]$$

The hidden state formula is:

$$\boxed{h_t = o_t \odot \tanh(C_t)}$$

Therefore:

$$[0.7, 0.6] \odot [0.4621, 0.0997]$$

Calculate:

$$0.7 \times 0.4621 = 0.32347$$

$$0.6 \times 0.0997 = 0.05982$$

Therefore:

$$\boxed{h_t \approx [0.3235, 0.0598]}$$

9. Complete calculation in one place

Given:

$$C_{t-1} = [0.6, 0.8] \quad f_t = [0.5, 0.25] \quad i_t = [0.4, 0.5]$$

$$\tilde{C}_t = [0.5, -0.2] \quad o_t = [0.7, 0.6]$$

Forget:

$$f_t \odot C_{t-1} = [0.5, 0.25] \odot [0.6, 0.8] = [0.3, 0.2]$$

Input $\times$ Candidate:

$$i_t \odot \tilde{C}_t = [0.4, 0.5] \odot [0.5, -0.2] = [0.2, -0.1]$$

New Cell State:

$$C_t = [0.3, 0.2] + [0.2, -0.1] = [0.5, 0.1]$$

Tanh:

$$\tanh(C_t) = \tanh([0.5, 0.1]) = [0.4621, 0.0997]$$

Hidden State:

$$h_t = o_t \odot \tanh(C_t) = [0.7, 0.6] \odot [0.4621, 0.0997] = [0.3235, 0.0598]$$

10. The entire process, visually

PREVIOUS MEMORY

$$C_{t-1} = [0.6, 0.8]$$

$$\begin{array}{c} \downarrow \\ \times \\ \downarrow \\ \text{FORGET GATE } [0.5, 0.25] \end{array}$$

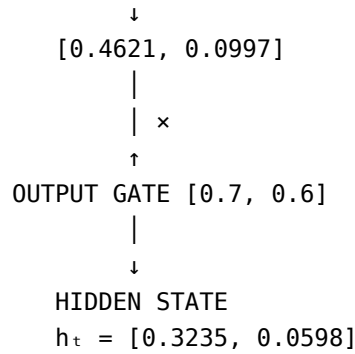
$$[0.3, 0.2]$$

$$\begin{array}{c} \downarrow \\ + \\ \uparrow \\ [0.2, -0.1] \\ \uparrow \\ \times \\ \downarrow \\ \text{INPUT GATE } [0.4, 0.5] \times \text{CANDIDATE } [0.5, -0.2] \end{array}$$

NEW CELL STATE

$$C_t = [0.5, 0.1]$$

$$\downarrow \\ \tanh$$



11. What each number means

Component	Value	Meaning
C_{t-1}	[0.6, 0.8]	Previous long-term memory
f_t	[0.5, 0.25]	How much old memory to keep
$f_t \odot C_{t-1}$	[0.3, 0.2]	Retained old memory
i_t	[0.4, 0.5]	How much new information to allow
$\tilde{C}_t$	[0.5, -0.2]	Candidate new information
$i_t \odot \tilde{C}_t$	[0.2, -0.1]	New information actually added
C_t	[0.5, 0.1]	New long-term memory
o_t	[0.7, 0.6]	How much memory to expose
h_t	[0.3235, 0.0598]	New hidden state

12. The key distinction

Notice this:

$$C_t = [0.5, 0.1]$$

but:

$$h_t = [0.3235, 0.0598]$$

They are **not the same thing**.

C_t

↓

Long-term memory

"What should I remember?"

h_t

↓

Current hidden state

"What should I expose / use right now?"

That's why LSTM has **two states**.

13. One-line memory trick

Remember the LSTM time step as:

$$\text{FORGET OLD} \rightarrow \text{ADD NEW} \rightarrow \text{CREATE } C_t \rightarrow \text{SELECT OUTPUT} \rightarrow \text{CREATE } h_t$$

Or even shorter:

Forget → Add → Remember → Output

And the two equations you absolutely must know are:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

and

$$h_t = o_t \odot \tanh(C_t)$$

Next step — go one level deeper: calculate where f_t , i_t , $\tilde{C}_t$, and o_t themselves come from, using actual x_t , h_{t-1} , weight matrices, biases, sigmoid, and tanh. That is the part that connects the LSTM equations to the actual neural-network computation.

- Calculate the gates from weights